INTRODUCTION:

This report will include the database design, the Entity-Relationship model and schema, and give an overview of how it is used to effectively analyze and collect the required data and relationships between entities. This report will also include the progress made in implementing our designed database, such as the creation of tables, defining suitable data types and establishing relationships between said tables. All efforts made to ensure data consistency, security and integrity through the implementation of constraints, normalization techniques and access control will be showcased.

SYSTEM OVERVIEW AND FUNCTIONAL DESCRIPTION:

This section provides an overview of the Sports Pavilion database system, highlighting its key functionalities and how different components interact to support real-world operations. It explains how the system manages users, facilities, bookings, payments, and staff roles in an organized and efficient manner.

The Sports Pavilion Database Project aims to design a comprehensive database system to efficiently manage the operations of a multi-activity sports center. This pavilion offers a variety of facilities including sports courts, gym services, and recreational activities for both members and visitors. The primary objective of the project is to organize and store large volumes of data related to users, facilities, bookings, and payments in a structured manner. The system distinguishes between registered members and walk-in visitors, allowing both to interact with the pavilion’s services while maintaining appropriate access controls. Members can subscribe to different membership plans, each offering specific benefits such as gym access and duration-based privileges.

The database also manages bookings for various play units, ensuring that facilities are allocated properly without conflicts or double reservations. Each play unit is associated with a specific sports facility, reflecting the real-world structure of courts, lanes, and playing areas. Payments made by members and visitors are tracked to maintain financial records and support transaction transparency. Additionally, staff members are included in the system to manage and maintain facilities, with specialized roles such as gym trainers responsible for handling equipment and assisting users.

The design incorporates strong and weak entity sets, along with relationships that accurately represent real-world interactions within the pavilion. Multi-valued and composite attributes are used to capture detailed personal information, while specialization helps in modeling staff roles

effectively. Overall, the project demonstrates how a well-structured database can support operational efficiency, data integrity, and scalability in a real-life business environment.

A. Requirements Formulation and Analysis

The requirements formulation and analysis phase involves collecting and analyzing information about the Sports Pavilion to identify the data and processing needs of the system. This includes understanding how different parts of the pavilion operate, such as sports facilities, gym services, bookings, payments, and staff management. Information was gathered by considering different user views and identifying the data items and relationships required for efficient operation.

Different **user views** were identified based on roles within the pavilion. For example, a **Manager** requires overall reports such as total bookings, revenue, and facility usage. A **Receptionist** handles visitor and member registrations, bookings, and payments, requiring access to data such as member details, booking schedules, and payment records. A **Gym Staff/Trainer** needs information about gym equipment and availability timings, while general **Visitors/Members** interact with booking and payment services. Each view defines specific data requirements such as member information, facility details, booking time slots, and payment records.

The system must support various **transactions**, including registering members and visitors, booking sports facilities, recording payments, and managing gym equipment and staff. Frequently occurring transactions include booking creation, payment processing, and checking facility availability. Reports such as daily bookings, monthly revenue, and equipment status are also required for smooth operation.

Additional system requirements include ensuring **data integrity and security**, such as preventing double bookings, maintaining valid payment records, and restricting gym access to members only. The system should also provide quick response times for booking and availability checks, along with basic protection against data loss. These requirements form the foundation for designing an efficient and reliable database system for the Sports Pavilion.

B. Conceptual Design

The conceptual design represents the overall structure of the database using an Entity-Relationship (E-R) model. Key entities such as Visitor, Member, Staff, Membership Plan, Sports Facility, Play Unit, Booking, Payment, Gym Equipment, and Gym Trainer were identified along with their attributes and primary keys. Relationships between these entities were defined to reflect real-world interactions, including one-to-many and many-to-many associations.

Special design features such as weak entities and specialization were also incorporated. The Play Unit entity was modeled as a weak entity dependent on Sports Facility, while Gym Trainer

was represented as a subtype of Staff using specialization. Appropriate constraints like cardinality and participation were considered to ensure that the model accurately reflects the system requirements.

C. Logical Design

In the logical design phase, the conceptual E-R model was converted into a relational schema consisting of tables, attributes, and keys. Each entity was transformed into a relation with a primary key, and relationships were implemented using foreign keys. One-to-many relationships were handled by placing foreign keys on the “many” side, while many-to-many relationships were resolved using separate tables where required.

Normalization techniques were applied to organize the data efficiently and eliminate redundancy. The schema was designed to satisfy Third Normal Form (3NF), ensuring minimal data duplication and preventing update anomalies. Special cases such as weak entities and specialization were properly handled using composite keys and subclass tables

D. Physical Design

The physical design phase involved implementing the logical schema using Oracle SQL. Tables were created using DDL statements, with appropriate data types assigned to each attribute. Constraints such as primary keys, foreign keys, NOT NULL, UNIQUE, and CHECK were applied to maintain data integrity and enforce business rules.

Indexes were created on important attributes like foreign keys and frequently queried fields to improve performance. Additional considerations such as time constraints in bookings and proper ownership of payments were incorporated to ensure correctness and efficiency. Overall, the physical design ensures that the database is optimized for real-world usage and performance.

RELATIONAL SCHEMA:

Strong Entity Sets;

- **VISITOR** (**visitor_id**, vis_phone, vis_name, visit_date)
- **MEMBER** (**member_id**, plan_id (FK), mem_name, mem_email, mem_join_date, mem_phone)
- **STAFF** (**staff_id**, staff_phone, staff_role, staff_salary, staff_name)
- **MEMBERSHIP_PLAN** (**plan_id**, plan_name, duration_months, plan_price, gym_access)
- **GYM_EQUIPMENT** (**equipment_id**, equipment_name, ge_brand, ge_purchase_date, ge_condition)
- **SPORTS_FACILITY** (**facility_id**, facility_name, facility_type, sp_hourly_rate)
- **PAYMENT** (**payment_id**, member_id (FK), visitor_id (FK), plan_id (FK), booking_id (FK), amount, payment_date, payment_method)
- **BOOKING** (**booking_id**, member_id (FK), visitor_id (FK), facility_id (FK), unit_num (FK), booking_date, bk_start_time, bk_end_time)

Weak Entity Sets;

- **PLAY_UNIT** (**facility_id**, unit_num, unit_status)

Specializations;

- **GYM_TRAINER** is a specialization of **STAFF**.
  - *Inherited Attributes:* (**staff_id**, staff_phone, staff_name, staff_salary)
  - *Specific Attributes:* (trainer_start_time, trainer_end_time)

Multi-Valued Attributes;

- **VISITOR:** vis_phone
- **MEMBER:** mem_phone.
- **STAFF:** staff_phone.
- **BOOKING:** facility_id, unit_num, bk_start_time and bk_end_time

Composite Attributes;

- **vis_name:** vis_first_name, vis_last_name
- **mem_name:** mem_first_name, mem_last_name
- **staff_name:** staff_first_name, staff_last_name

Relationships;

- **PAYS:** Connects **VISITOR** to **PAYMENT** (1:N).
- **MAKES:** Connects **VISITOR** to **BOOKING** (1:N).
- **MAKES:** Connects **MEMBER** to **BOOKING** (1:N).
- **PAYS:** Connects **MEMBER** to **PAYMENT** (1:N).
- **SUBSCRIBES:** Connects **MEMBER** to **MEMBERSHIP_PLAN** (N:1).
- **RESERVES:** Connects **BOOKING** to **PLAY_UNIT** (N:1).
- **BELONGS_TO:** Connects **PLAY_UNIT** to **SPORTS_FACILITY** (N:1).
- **MAINTAINS:** Connects **STAFF** to **SPORTS_FACILITY** (M:N).
- **MAINTAINS:** Connects **GYM_TRAINER** to **GYM_EQUIPMENT** (M:N).
- **ISA:** Connects **GYM_TRAINER** to **STAFF** (Supertype/Subtype relationship).

NORMALIZED TUPLES;

- **VISITOR** (**visitor_id**, vis_name, visit_date)
- **VIS_PHONE** (visitor_id(FK), vis_phone)
- **MEMBER** (**member_id**, plan_id (FK), mem_name, mem_email, mem_join_date)
- **MEMBER_PHONE** (**member_id**, mem_phone)
- **STAFF** (**staff_id**, staff_role, staff_salary, staff_name)
- **STAFF_PHONE** (**staff_id**, staff_phone)
- **MEMBERSHIP_PLAN** (**plan_id**, plan_name, duration_months, plan_price, gym_access)
- **GYM_EQUIPMENT** (**equipment_id**, equipment_name, ge_brand, ge_purchase_date, ge_condition)
- **SPORTS_FACILITY** (**facility_id**, facility_name, facility_type, sp_hourly_rate)
- **PAYMENT** (**payment_id**, member_id (FK), visitor_id (FK), plan_id (FK), booking_id (FK), amount, payment_date, payment_method)
- **MEMBERSHIP_PAYMENT** (**payment_id**, member_id (FK), plan_id (FK))
- **BOOKING_PAYMENT** (**payment_id**, booking_id (FK))
- **BOOKING** (**booking_id**, member_id (FK), visitor_id (FK), facility_id (FK), unit_num (FK), booking_date, bk_start_time, bk_end_time)
- **MEMBER_BOOKING** (**booking_id**, member_id (FK))
- **VISITOR_BOOKING (booking_id**, visitor_id (FK))
- **PLAY_UNIT** (**facility_id**, unit_num, unit_status)

DDL Statements For Tables;

1. MEMBERSHIP PLAN

```
CREATE TABLE MembershipPlan (
    plan_id INT AUTO_INCREMENT PRIMARY KEY,
    plan_name VARCHAR(100) NOT NULL,
    duration_months INT UNSIGNED NOT NULL,
    plan_price DECIMAL(10,2) NOT NULL,
    gym_access BOOLEAN DEFAULT TRUE
);
```

2. SPORTS FACILITY

```
CREATE TABLE SportsFacility (
    facility_id INT AUTO_INCREMENT PRIMARY KEY,
    facility_name VARCHAR(100) NOT NULL,
    facility_type VARCHAR(50) NOT NULL,
    sp_hourly_rate DECIMAL(8,2) NOT NULL
);
```

3. GYM EQUIPMENT

```
CREATE TABLE GymEquipment (
    equipment_id INT AUTO_INCREMENT PRIMARY KEY,
    equipment_name VARCHAR(100) NOT NULL,
    ge_brand VARCHAR(50) NOT NULL,
    ge_purchase_date DATE NOT NULL,
    ge_condition VARCHAR(50) NOT NULL
);
```

4. PLAY UNIT (WEAK ENTITY)

```
CREATE TABLE PlayUnit (
    unit_id INT AUTO_INCREMENT PRIMARY KEY,
    facility_id INT NOT NULL,
    unit_num VARCHAR(20) NOT NULL,
    unit_status VARCHAR(20) DEFAULT 'Available',

    CONSTRAINT fk_playunit_facility
        FOREIGN KEY (facility_id)
        REFERENCES SportsFacility(facility_id)
        ON DELETE CASCADE,

    CONSTRAINT uq_facility_unit
        UNIQUE (facility_id, unit_num)
);
```

5. STAFF

```
CREATE TABLE Staff (
   staff_id INT AUTO_INCREMENT PRIMARY KEY,
   staff_first_name VARCHAR(50) NOT NULL,
   staff_last_name VARCHAR(50) NOT NULL,
   staff_role VARCHAR(50) NOT NULL,
   staff_salary DECIMAL(10,2) NOT NULL
);
```

6. STAFF ↔ SPORTS FACILITY (M:N)

```
CREATE TABLE Staff_Maintains_Facility (
   staff_id INT NOT NULL,
   facility_id INT NOT NULL,

   PRIMARY KEY (staff_id, facility_id),

   CONSTRAINT fk_staffmaint_staff
      FOREIGN KEY (staff_id)
      REFERENCES Staff(staff_id)
      ON DELETE CASCADE,

   CONSTRAINT fk_staffmaint_facility
      FOREIGN KEY (facility_id)
      REFERENCES SportsFacility(facility_id)
      ON DELETE CASCADE
);
```

7. GYM TRAINER (SPECIALIZATION OF STAFF)

```
CREATE TABLE GymTrainer (
   staff_id INT PRIMARY KEY,
   trainer_start_time TIME NOT NULL,
   trainer_end_time TIME NOT NULL,

   CONSTRAINT fk_trainer_staff
      FOREIGN KEY (staff_id)
      REFERENCES Staff(staff_id)
      ON DELETE CASCADE
);
```

```
-- 8. TRAINER ↔ EQUIPMENT (M:N)

CREATE TABLE Trainer_Maintains_Equipment (
   staff_id INT NOT NULL,
   equipment_id INT NOT NULL,

   PRIMARY KEY (staff_id, equipment_id),

   CONSTRAINT fk_trainermaint_trainer
      FOREIGN KEY (staff_id)
```

```
    REFERENCES GymTrainer(staff_id)
    ON DELETE CASCADE,

  CONSTRAINT fk_trainermaint_equipment
    FOREIGN KEY (equipment_id)
    REFERENCES GymEquipment(equipment_id)
    ON DELETE CASCADE
);
```

9. VISITOR

```
CREATE TABLE Visitor (
  visitor_id INT AUTO_INCREMENT PRIMARY KEY,
  vis_first_name VARCHAR(50) NOT NULL,
  vis_last_name VARCHAR(50) NOT NULL,
  visit_date DATE NOT NULL DEFAULT (CURRENT_DATE)
);
```

10. MEMBER

```
CREATE TABLE Member (
  member_id INT AUTO_INCREMENT PRIMARY KEY,
  plan_id INT NULL,
  mem_first_name VARCHAR(50) NOT NULL,
  mem_last_name VARCHAR(50) NOT NULL,
  mem_email VARCHAR(254) NOT NULL UNIQUE,
  mem_join_date DATE NOT NULL,

  CONSTRAINT fk_member_plan
    FOREIGN KEY (plan_id)
    REFERENCES MembershipPlan(plan_id)
    ON DELETE SET NULL
);
```

11. VISITOR PHONE

```
CREATE TABLE VisitorPhone (
  id INT AUTO_INCREMENT PRIMARY KEY,
  visitor_id INT NOT NULL,
  phone_number VARCHAR(15) NOT NULL,

  CONSTRAINT fk_visitorphone_visitor
    FOREIGN KEY (visitor_id)
    REFERENCES Visitor(visitor_id)
    ON DELETE CASCADE
);
```

12. MEMBER PHONE

```
CREATE TABLE MemberPhone (
   id INT AUTO_INCREMENT PRIMARY KEY,
   member_id INT NOT NULL,
   phone_number VARCHAR(15) NOT NULL,

   CONSTRAINT fk_memberphone_member
      FOREIGN KEY (member_id)
      REFERENCES Member(member_id)
      ON DELETE CASCADE );
```

13. STAFF PHONE

```
CREATE TABLE StaffPhone (
   id INT AUTO_INCREMENT PRIMARY KEY,
   staff_id INT NOT NULL,
   phone_number VARCHAR(15) NOT NULL,

   CONSTRAINT fk_staffphone_staff
      FOREIGN KEY (staff_id)
      REFERENCES Staff(staff_id)
      ON DELETE CASCADE
);
```

14. BOOKING

```
CREATE TABLE Booking (
   booking_id INT AUTO_INCREMENT PRIMARY KEY,
   unit_id INT NOT NULL,
   booking_date DATE NOT NULL,
   bk_start_time TIME NOT NULL,
   bk_end_time TIME NOT NULL,

   CONSTRAINT fk_booking_playunit
      FOREIGN KEY (unit_id)
      REFERENCES PlayUnit(unit_id)
      ON DELETE CASCADE
);
```

15. MEMBER BOOKING

```
CREATE TABLE MemberBooking (
   booking_id INT PRIMARY KEY,
   member_id INT NOT NULL,

   CONSTRAINT fk_memberbooking_booking
      FOREIGN KEY (booking_id)
      REFERENCES Booking(booking_id)
      ON DELETE CASCADE,
```

```
    CONSTRAINT fk_memberbooking_member
        FOREIGN KEY (member_id)
        REFERENCES Member(member_id)
        ON DELETE CASCADE
);
```

16. VISITOR BOOKING

```
CREATE TABLE VisitorBooking (
    booking_id INT PRIMARY KEY,
    visitor_id INT NOT NULL,

    CONSTRAINT fk_visitorbooking_booking
        FOREIGN KEY (booking_id)
        REFERENCES Booking(booking_id)
        ON DELETE CASCADE,

    CONSTRAINT fk_visitorbooking_visitor
        FOREIGN KEY (visitor_id)
        REFERENCES Visitor(visitor_id)
        ON DELETE CASCADE
);
```

17. PAYMENT

```
CREATE TABLE Payment (
    payment_id INT AUTO_INCREMENT PRIMARY KEY,
    amount DECIMAL(10,2) NOT NULL,
    payment_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    payment_method VARCHAR(50) NOT NULL
);
```

18. MEMBERSHIP PAYMENT

```
CREATE TABLE MembershipPayment (
    payment_id INT PRIMARY KEY,
    member_id INT NOT NULL,
    plan_id INT NOT NULL,

    CONSTRAINT fk_membershippayment_payment
        FOREIGN KEY (payment_id)
        REFERENCES Payment(payment_id)
        ON DELETE CASCADE,

    CONSTRAINT fk_membershippayment_member
        FOREIGN KEY (member_id)
        REFERENCES Member(member_id)
        ON DELETE CASCADE,

    CONSTRAINT fk_membershippayment_plan
        FOREIGN KEY (plan_id)
```

```
    REFERENCES MembershipPlan(plan_id)
    ON DELETE CASCADE
);
```

19. BOOKING PAYMENT

```
CREATE TABLE BookingPayment (
  payment_id INT PRIMARY KEY,
  booking_id INT NOT NULL,

  CONSTRAINT fk_bookingpayment_payment
    FOREIGN KEY (payment_id)
    REFERENCES Payment(payment_id)
    ON DELETE CASCADE,

  CONSTRAINT fk_bookingpayment_booking
    FOREIGN KEY (booking_id)
    REFERENCES Booking(booking_id)
    ON DELETE CASCADE
);
```

What is Django?

Django is a high-level open-source Python web framework designed for rapid development of secure and maintainable web applications. It follows the **"batteries included"** philosophy, meaning it provides everything needed to build a web application out of the box without relying on external tools. Django was first released in 2005 and is maintained by the Django Software Foundation.

In our Sports Pavilion project, Django serves as the complete backend framework connecting the user interface to the MySQL database.

Django's Architecture — MTV Pattern

Django follows the **Model-Template-View (MTV)** pattern which is similar to the traditional MVC (Model-View-Controller) pattern.

MTV Component	Role in Django	Our Project Example
Model	Defines database structure	`Member`, `Booking`, `Payment` classes
Template	HTML presentation layer	`index.html`, `login.html`, `member_list.html`
View	Business logic	`loginUser()`, `book_member()`, `become_member()`

Main Components Used in Our Project

---

1. Models (`models.py`)

Models are Python classes that define the structure of database tables. Django's ORM (Object Relational Mapper) automatically converts these classes into MySQL tables.

```python
class Member(models.Model):
    member_id = models.AutoField(primary_key=True)
    # SUBSCRIBES relationship (N:1 Member → MembershipPlan)
    plan = models.ForeignKey(MembershipPlan, on_delete=models.SET_NULL, null=True, blank=True, related_name='members')
    # Composite attribute (mem_name) flattened:
    mem_first_name = models.CharField(max_length=50)
    mem_last_name = models.CharField(max_length=50)
    mem_email = models.EmailField(unique=True)
    mem_join_date = models.DateField()
```

In our project we defined these models:

- `Member`, `Visitor`, `Staff`, `GymTrainer`
- `MembershipPlan`, `SportsFacility`, `PlayUnit`
- `Booking`, `MemberBooking`, `VisitorBooking`
- `Payment`, `BookingPayment`, `MembershipPayment`
- `GymEquipment`, `MemberPhone`, `VisitorPhone`, `StaffPhone`

---

2. Views (`views.py`)

Views contain the business logic of the application. They receive web requests, process data, interact with models, and return responses to the user.

```python
def loginUser(request):
    if request.user.is_authenticated:
        return redirect('home')

    if request.method == 'POST':
        u_name = request.POST.get('username').strip()
        p_word = request.POST.get('password')
        user = authenticate(username=u_name, password=p_word)

        if user is not None:
            login(request, user)
            request.session.set_expiry(1209600) # 2 weeks in seconds

            if user.is_staff:
                return redirect('member_list') # Redirect Admin to /members
            else:
                messages.success(request, f"Welcome {user.username}!")
                return redirect('home') #return to index page for regular users
        else:
            context = {'title': 'Login'}
            messages.error(request, "Invalid username or password")
            return render(request, 'login.html', context)

    context = {'title': 'Login'}
    return render(request, 'login.html', context)
```

Main views in our project:

- `index` — Public home page
- `loginUser` — Handles login/logout
- `registerUser` — Creates new accounts
- `become_member` — Membership registration
- `book_visitor` — Visitor booking
- `book_member` — Member booking
- `member_list` — Admin member management

---

3. URLs (`urls.py`)

The URL configuration maps web addresses to their corresponding view functions.

```
urlpatterns = [
    path('register', views.registerUser, name='register'),
    path('', views.index, name='home'),
    path('about', views.about, name='about'),
    path('login', views.loginUser, name='login'),
    path('logout', views.logoutUser, name='logout'),
    path('book-visitor', views.book_visitor, name='book_visitor'),
    path('become-member', views.become_member, name='become_member'),
    path('book-member', views.book_member, name='book_member'),
```

---

4. Templates (`templates/`)

Templates are HTML files that display data to the user. Django uses its own templating language to inject dynamic data into HTML pages.

{% if request.user.is_authenticated %}
<h5>Welcome, {{ request.user.username }}!</h5>
{% else %}
Login
{% endif %}

Templates in our project:

- `index.html` — Public home page
- `login.html` — Sign in and sign up
- `member_list.html` — Admin member management
- `member_profile.html` — Individual member details
- `book_visitor.html` — Visitor booking form
- `book_member.html` — Member booking form
- `become_member.html` — Membership registration form

---

5. Django Admin Panel

Django automatically generates a powerful admin interface for managing all database records without writing any extra code.

In our project the admin panel allows:

- Managing members and visitors
- Viewing all bookings
- Managing membership plans
- Monitoring payments
- Managing sports facilities

---

6. Authentication System

Django provides a built-in authentication system that handles user registration, login, logout, and session management securely.

user = authenticate(username=u_name, password=p_word)
login(request, user)
request.session.set_expiry(1209600) # 2 weeks

In our project:

- Passwords are automatically **hashed** for security
- Sessions persist for **2 weeks**
- `@login_required` decorator protects sensitive pages
- Admin and normal users have different access levels

---

7. ORM (Object Relational Mapper)

Django's ORM allows us to interact with the MySQL database using Python code instead of writing raw SQL queries.

SQL Query	Django ORM Equivalent
`SELECT * FROM member`	`Member.objects.all()`
`INSERT INTO member...`	`Member.objects.create(...)`
`UPDATE member SET...`	`member.save()`
`DELETE FROM member...`	`member.delete()`
`SELECT WHERE email=...`	`Member.objects.filter(mem_email=email)`

---

8. Migrations

Migrations are Django's way of tracking and applying changes to the database schema. Every time models are changed, migrations are created and applied.

bash
```
python manage.py makemigrations  # Creates migration files
python manage.py migrate         # Applies changes to MySQL
```

9. Middleware & Security

Django provides built-in security features used in our project:

Security Feature	Purpose
CSRF Protection	Prevents cross-site request forgery attacks on forms
Password Hashing	Passwords stored securely using PBKDF2 algorithm
Session Management	Keeps users logged in securely
SQL Injection Prevention	ORM automatically sanitizes all queries

Why Django for This Project?

Requirement	How Django Helped
Database management	ORM connected seamlessly to MySQL
User authentication	Built-in auth system saved development time
Admin panel	Auto-generated for managing all data
Security	Built-in CSRF, password hashing, SQL injection prevention
Rapid development	Reduced development time significantly
Clean code structure	MTV pattern kept code organized